ExactDomainEvaluationNet: Investigating Convolutional Machine Learning Applied to the LUX-ZEPLIN Experiment's Photon Counter

Elies Ducluzeau

April 2026

Abstract

This project focuses on the LUX-ZEPLIN experiment for dark matter detection, particularly the existing conventional algorithms used to interpret events, de-noise signals, and count photons detected from WIMP nuclear recoils. This method of searching for WIMPs has begun to encounter limitations owing to factors such as noise, signal smearing, and the immitigable unknowns involved in the task. To continue reducing the uncertainty in the detector's measurements, we investigate machine learning and attempt to apply it to LZ photon counting by investigating ExactDomainEvaluationNet, a machine learning model constructed to classify photon events using a pre-existing LZ event simulation program for training and evaluation.

Contents

1 Introduction **2**
- 1.1 Dark Matter . . . 2
- 1.2 The LUX-ZEPLIN Experiment . . . 2
- 1.3 Photon Counting and Reconstruction . . . 4
- 1.4 LUX-ZEPLIN Dark Matter Leakage . . . 5
- 1.5 Preliminary Machine Learning . . . 6

2 Peak Finding and Conventional Programming **6**
- 2.1 Photon Emission . . . 6
- 2.2 Photon Counting Simulation . . . 8
- 2.3 Leakage Plotting in Python . . . 8
- 2.4 Peak Finding in Python . . . 9

3 Machine Learning **11**
- 3.1 Neural Networks . . . 11
- 3.2 Training And Evaluating Neural Networks . . . 12
- 3.3 ResNet and RawNet . . . 14

3.4 RawNet For Particle Physics 16
3.5 Adapting RawNet to This Problem 17

4 Performance **20**
4.1 Conventional 20
4.2 EDEN 22
4.3 Comparisons and Progression 23

5 Conclusions **25**

A Acknowledgements **29**

1 Introduction

1.1 Dark Matter

Dark matter (DM) is a hypothesised form of matter which is thought to exist due to separate gravitational anomalies observed at all scales of cosmology. The Coma Cluster was shown to be rotating faster than Keplerian predictions would allow for[1], which was later extended to all galaxies[2]. Anisotropies in the cosmic microwave background show signs of influence from more early-universe gravitation than Standard Model models allow for[3], and bodies such as the Bullet Cluster show direct evidence of non-visible gravitation[4].

A potential solution to this problem is weakly interacting massive particles (WIMPs)[5], which are relativistic particles that only interact via gravity and very rarely via the weak force. If these particles exist, it is believed that they would cluster in massive rotating structures around galaxies known as "halos", which would sometimes intersect the Earth and allow for a direct detection of DM. The main challenge posed by this solution is that, since the local density of the Milky Way's dark matter halo in the Solar System is predicted to be approximately 0.3 ± 0.03 GeVcm^{-3}[6][7], they are predicted to interact rarely enough via the weak force for them to overall appear to not interact beyond gravitation, which sets harsh guidelines on what a DM experiment must do to potentially directly detect a WIMP.

1.2 The LUX-ZEPLIN Experiment

The LUX-ZEPLIN (LZ) experiment is a 7t liquid xenon (LXe) WIMP direct detection experiment.[8] This experiment aims to directly detect a passing WIMP via a weak interaction event with a LXe nucleus located in the detector's time projection chamber, which would cause the nucleus to recoil and emit scintillation photons in addition to an ionisation electron[9].

LXe is chosen because of its potent scintillation, which produces a prominent photonic signal that can be detected by the large array of photomultiplier tubes (PMTs) at the base of the time projection chamber. Across the time projection chamber, a weak electric field is applied such that the ionisation electron drifts towards the top of the chamber,

where a gaseous xenon (GXe) block decelerates the electron, causing a photon shower via electroluminescence. These photons are detected by a separate array of PMTs at the top of the chamber. PMTs are devices that use the photoelectric effect to record an incoming photon as an electronic signal as a plot of amplitude against time (a "time-domain waveform"); these signals are used to reconstruct the progression of the event.[10]

Owing to the rarity of dark matter interactions, de-noising is the priority for the experiment. To this end, the time projection chamber is enclosed in a cryostat vessel with a "skin" of further LXe inside, which act as a veto to incoming neutrons and gamma radiation. The detector itself is constructed out of highly radiopure materials to avoid contamination from its own components, and it is located a mile underground at the Sanford Underground Research Facility to filter out cosmic rays.

Neutrinos are still capable of penetrating this material and interacting with the LXe, acting as the main source of noise for the detector and putting a lower bound on possible WIMP detections known as the "neutrino fog"[11]. So far even abiding by these limitations, LZ and other dark matter experiments have discovered the following constraints for the WIMP cross-section:

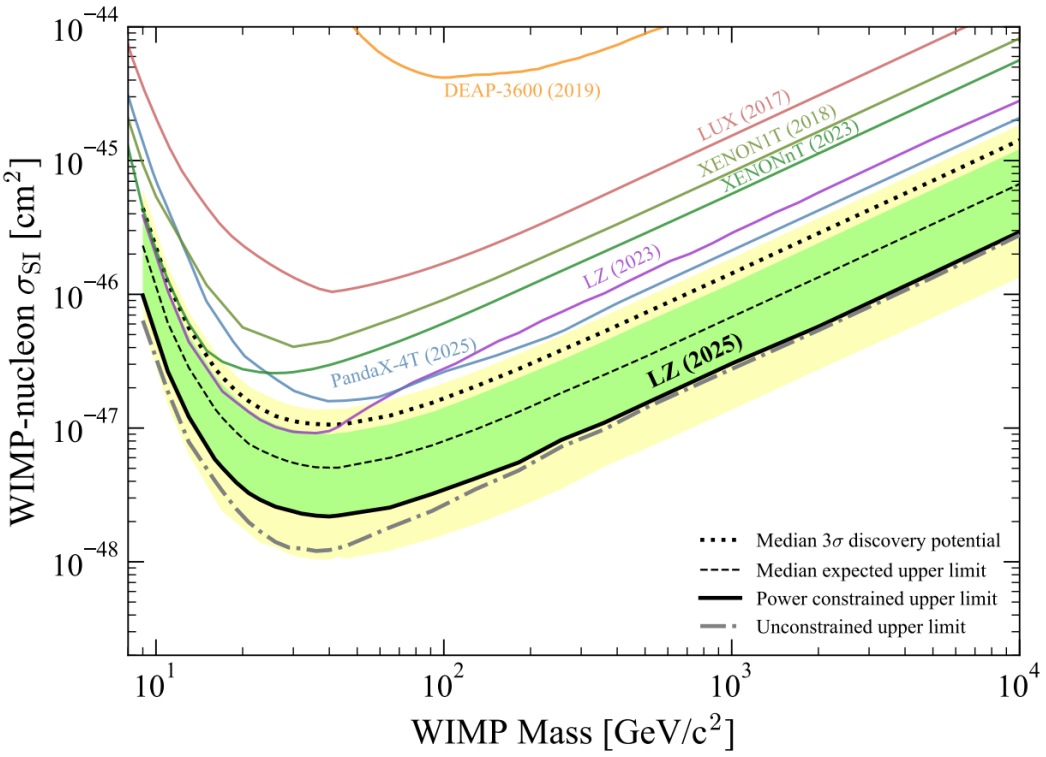


Figure 1: Upper limits (90% C.L.) on the spin-independent WIMP-nucleon cross section as a function of WIMP mass from the combined WS2024+WS2022 analysis (280.0 live days) from LZ runs, as well as data gathered from PandaX-4T, DEAP-3600, LUX, and XENON.[12]

1.3 Photon Counting and Reconstruction

To obtain data from events, their details must be reconstructed from the parameters of the photons detected by the PMTs. This is done by detecting the interval between the arrival of the scintillation signal (the "S1 signal") and the photon cascade from the ionisation electron (the "S2 signal") to determine the event's vertical position, while determining the planar position of the event by seeing which batch of PMTs detected it the most.

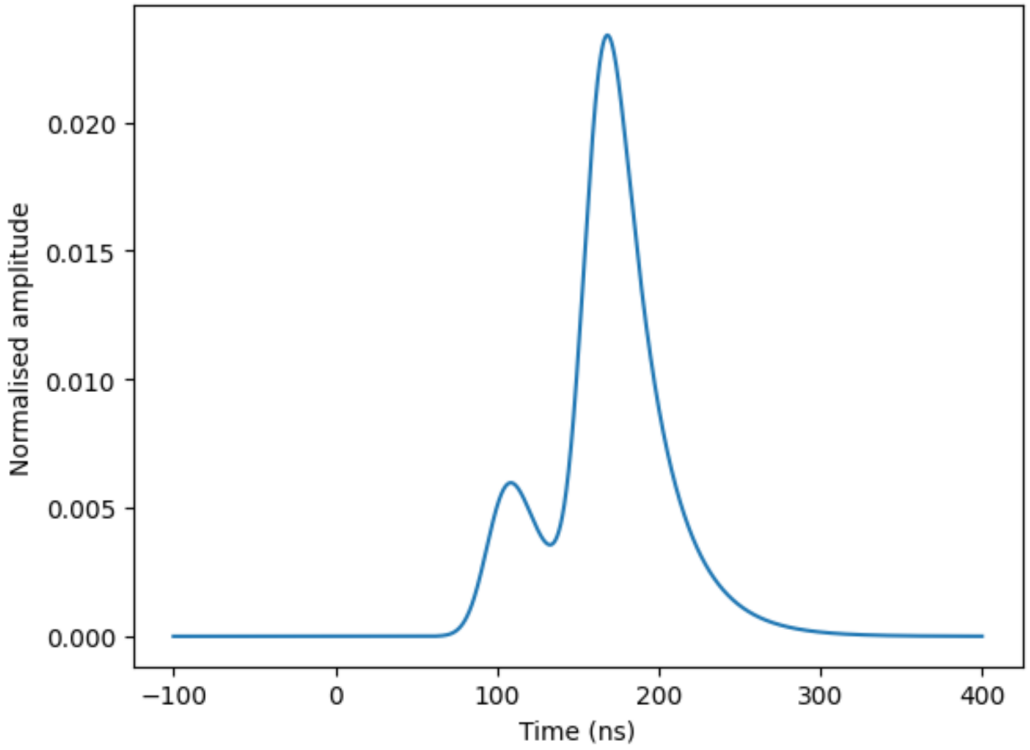


Figure 2: An example of the resultant S1 time-domain waveform returned from one PMT.[Made in Python]

Owing to the output of a PMT being electronic in nature, the data from a PMT is displayed as a time-domain waveform. In the context of LZ, information is extracted from these waveforms by determining the amount of peaks present. Then, tracing photon detections across the whole PMT array and the time distance between the S1 and S2 signals returns a complete 3D picture of the entire event. Additionally, because PMTs use the photoelectric effect by detecting the photon and converting it into an electronic signal, they often randomly lose electrons before the electrons reach the detection component. This statistic is tracked as the PMT's "quantum efficiency". If the quantum efficiency of a given PMT in the bottom or top array respectively is known, the total event energy deposited by the initiating particle can be approximately found using the formula:

$$E = \left(\frac{S_1}{g_1} + \frac{S_2}{g_2} \right) \text{keV} \tag{1}$$

where the S_1 is the total number of detected scintillation photons, S_2 is the total number of detected electroluminescence photons, and g_1 and g_2 are the quantum efficiencies of the S1 and S2 detecting PMTs, respectively.

Several effects degrade the quality of the data obtained from PMTs. In addition to losses from quantum inefficiency, the neutrino fog and the instrument's own mechanical noise combine to add a significant noise term to the resulting photon detection. Additionally, charge smearing arises in the PMT due to the stochastic nature of the photoelectric effect[13] and the fact that photon arrival is spread out over 100 350 nanoseconds, which causes even singular photons to be detected as a distribution rather than a peak.

When put together, there is significant challenge to event reconstruction. Attempting to overcome these factors is usually done via conventional line by line programming of algorithms which can analyse event data and match it against criterion such as maxima in the differential of the data, distance between perceived maxima, or distance of the amplitude from zero, with the goal of seeing how many peaks exist in the resulting distribution. However, overlapping detections or low-intensity photons that exist close to the neutrino fog hamper such efforts. Conventional peak finding algorithms can be refined, but a feasible limit exists considering the unknowns driving the problem.

1.4 LUX-ZEPLIN Dark Matter Leakage

Alongside nuclear recoils (NRs) from WIMPs and neutrinos, LZ is also sensitive to electronic recoils (ERs) from various sources such as neutrino-electron scattering or trace detector radioactivity. This interaction produces a signal in the detector distinct from that of a nuclear recoil, however on occasion the emitted light from such an event may closely resemble the signature of a NR event.

To probe the detector's response to such events, calibration runs were conducted where the detector was dosed with tritium-tagged methane (CH3T) to provoke beta-decays and cause ER signals, and deuterium-deuterium (DD) fusion was used to fire neutrons into the detector, causing NR signals. Using the data from these trial runs, scenarios relating to challenges LZ could face in practice can be constructed.

In order to eliminate possible confusion when a signal is detected, it is imperative to maximize discrimination between NR and ER events, which can be accomplished via honing the detector's ability to count the amount of photons involved in a given event to distinguish the source. The fraction of ER events mistaken as NR events per a certain run of the experiment is known as the "leakage". Using the CH3T and DD datasets representing ER and NR events respectively, a visualisation of the leakage concept can be plotted. The graph is that of the area of the S2 signal against the area of the S1 signal for each event in both data sets (graph redacted for data confidentiality purposes).

Currently, LZ uses an algorithm that sums the area under the curve of the waveform to count photons. This is a continuous analysis of a discrete quantity, which smears data and may lead to the apparent leakage being greater than the true leakage; furthermore, this smearing may lead to miscounting of photons, which as per Equation 1 would distort the calculated energy deposited by the initiating particle in an event. Therefore, in the

interest of minimizing leakage and accurately reconstructing energy, it is necessary to push the boundaries of photon counting beyond conventional programming.

1.5 Preliminary Machine Learning

A machine learning (ML) model is a collection of programs that can analyse a data set through conventional routines which attempt to identify patterns present in these data sets. A ML model iterates through a data set, representing some concept in mathematical form, repeatedly and applies a series of matrix operations to it in an attempt to learn the patterns underlying the presented data. The model has some form of long-term "memory" of the analysed data which is applied for each re-run of the data set, allowing the model to self-improve. This is usually accomplished by giving each part of the model a "weight", which determines the magnitude of impact that each individual routine should have on the overall result. The self-improvement is achieved by modifying the weights given to each step of the identification process according to how close the model's results were to the true values. This allows a machine learning model to become better suited to the task it is being trained for the more time passes, eventually becoming able to approach unseen problems of the same type after having analysed enough training data. However, ML models can face problems in training that conventional programming usually does not face, one example being "bias", which is the ML terminology for any form of systematic error that can affect how the model learns. Bias can arise from such things as a human data scientist's own bias being reflected in the data they gather, to the use of a model too simplistic for a given task which leads to overgeneralization. To design a good ML model, it is important to find the best combination of weights for the task at hand while minimizing all forms of bias and finding ways to mitigate the challenges specific to the type of ML model being designed.[14]

Machine learning models are excellent tools for pattern recognition and classification, which are tasks where a large amount of data may or may not contain patterns which are impractical to conventionally determine for one reason or another. Such tasks include things like mathematical modelling and data reconstruction, which is the problem LZ has to solve. In recent times, ML models have been trialled in simulated problems using data sets from other experiments as training data.

A particularly pertinent one to LZ is the modified RawNet vocoder model trialled on JUNO data, which also deals in photon-event reconstruction from PMTs. This modified RawNet was able to discriminate between 0 through 9 photons in the analysis of a given PMT, and its creators note that the model may be usable for other photon counting problems. It appears to be a promising start, and is the model that will be investigated in this paper to improve LZ.

2 Peak Finding and Conventional Programming

2.1 Photon Emission

When a WIMP excites a LXe nucleus through the weak interaction, the nucleus emits scintillation photons in all directions[9], sourcing the first half of an LZ detection signal

which is known as the "S1 signal". The S1 signal is a sum time-domain waveform of all of the scintillation photons detected by the entire PMT array, each of which detect only a fraction of the emitted photons and display this information as their own sum time-domain waveforms; these waveforms represent noise, with amplitude peaks occurring where a photon is detected. Additionally, an ionization electron is emitted and drifts towards the GXe layer, undergoing electroluminescence on contact with this layer. The sum total of the electroluminescence photons for each PMT yields a signal known as the "S2 signal"; this is the second half of a LZ detection. In the LZ detector, PMT readouts are done as a time-

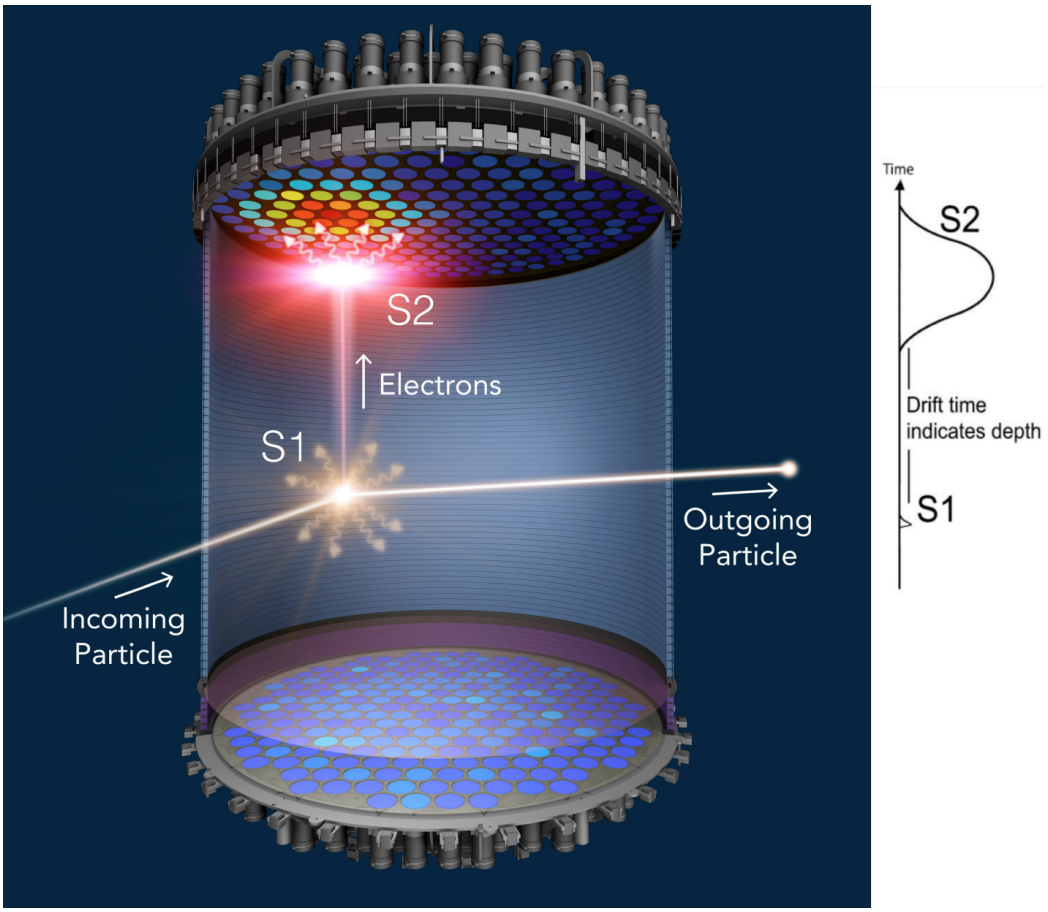


Figure 3: An example NR event in the LZ time projection chamber.[10]

domain amplitude waveform on an oscilloscope. For each PMT, both halves superpose on the oscilloscope readout to form a single waveform containing all of the interaction's information. The time delay between the S1 and S2 photons returns the vertical depth in the LXe tank where the WIMP caused a nucleus to recoil, while the proportion of signals detected by each PMT in the top and bottom arrays return the position of the event in the xy plane. The width of the signal can also partly return how strong the electroluminescence was. The total number of detected photons can be used as per Equation 1 to find the total energy of the incoming WIMP.

However, challenges exist to this approach. The sum waveform of every detected photon

at once would be almost unreadable, so instead the waveforms per each PMT are analysed. In order to extract information from these contributor waveforms, they must be broken down into the amount of photons that were detected and constructively interfered to cause them. Sometimes this number is visually apparent from a plot, but other times the readouts overlap and interfere constructively such that there appear to be less peaks than the number of contributing PMTs. This interference can additionally obfuscate information such as the kinetic energy of the incoming WIMP. Programs must be developed to circumvent these limitations.

2.2 Photon Counting Simulation

To prepare the LZ experiment for operation, improve its software components, and test theories and expectations, simulated scenarios are used in place of real trials. Owing to the complexity of the experiment, each part of LZ is engineered or simulated separately. For this project specifically, the simulation of S1 scintllations and their waveforms are the most relevant part of the experiment. Python and C are the main programming languages used for this task in LZ; this project uses Python. A custom "photon-counter" (PC) module for Python developed by Albert Baker allows for the simulation of WIMP events using Python's NumPy, hist, and Awkward modules; NumPy[15] allows the creation of arrays with a collection of built-in functions for data manipulation, Awkward[16] is a module for array creation that can connect to NumPy and the C language, and hist[17] is a specialised package for easy and detailed histogram creation. The PC module provides functions that create statistical models of LZ functions such as LXe scintillation or the delay taken for a photon to physically travel to a PMT, and functions that allow sampling of these statistical models according to a user-defined count of events to simulate and number of PMTs to sample events for.

In order to properly simulate events akin to what would occur for real in LZ, a template must be chosen for the behaviour of the simulated photons. For this project, an exponentially shifted Gaussian distribution was chosen as the baseline photon behaviour, to which the simulations and sampling were applied, as well as simulated stochastic noise to represent a neutrino fog. The result of this sequence is a NumPy array of raw data representing the net simulated amplitudes after summing up the simulated contributions from the accumulated photons and PMTs in the time-domain, which can then be plotted in various forms such as histograms or waveforms.

2.3 Leakage Plotting in Python

Once a data set is acquired through either a real experiment run or a simulation run, the leakage can be found algorithmically. The bulk LZ data must be run through Awkward, where it is screened to keep only the signal areas. Then, the hist module is used to bin the data along the S1 axis (graph redacted for data confidentiality purposes).

For each bin, the sum total of all CH3T data point locations in S2-S1 space are either represented as a near-Gaussian distribution or summed together, and for the DD space the data point locations are either represented as a near-Gaussian distribution or their median

is found. For the first method, the total area under the curves where both areas intersect yields the leakage for the given bin. For the second method, the proportion of the total CH3T residing above this DD median is found and expressed as a percentage; this gives the leakage for that given bin, and by plotting a graph of leakage against the bin, a profile for the overall leakage of a given experiment run is found. This leakage represents the uncertainty in telling apart a real event from a calibration signal.
For the leakage, the Clopper-Pearson[18] confidence interval is used to determine the statistical error as per the standard. The SciPy Python module, which provides various functions tailored towards scientific research, offers its own implementation of the Clopper-Pearson confidence interval. For this plot, the interval is taken to 1 sigma also as per the standard, and displayed as the error bar for each point.

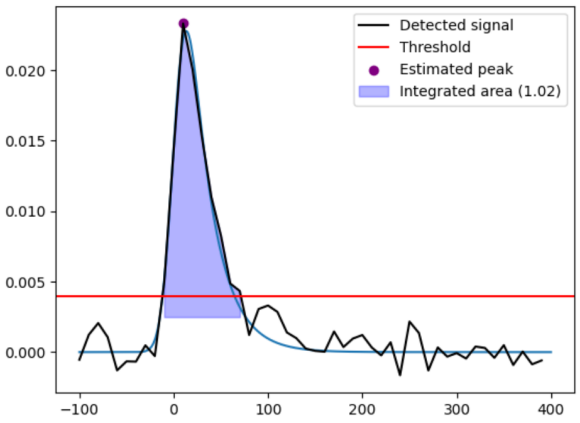
2.4 Peak Finding in Python

Peak finding is the act of analysing simulated or real PMT data in order to determine how many peaks, and therefore activated PMTs, caused the given event. LZ processes hundreds of thousands of data points, so analysing it all by human eye is impossible. This necessitates computation. However, a computer does not have the context to scrutinise graphs like humans do, requiring mathematical analysis on a low level. To do this, peak finding algorithms are employed, which check the raw NumPy data outputted by the simulator against a set of criteria, defined differently per algorithm. As part of the project, a simulator and peak finding algorithm were developed.

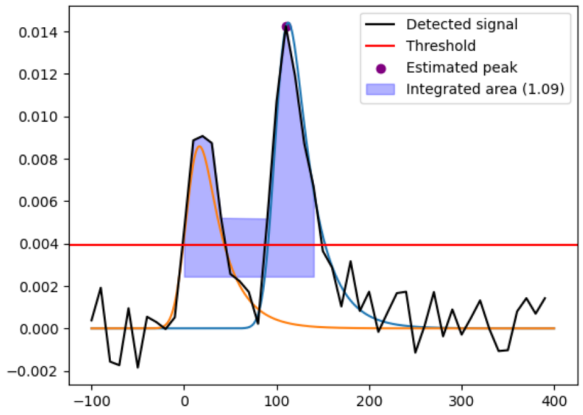
The algorithm first generates the waveforms by taking, for 1000 events and anywhere between 0 and 7 PMTs per event, an exponentially shifted Gaussian as the photon template behaviour. It then simulates the nuclear recoil electroluminescence responses, and time of flight delay in the time projection chamber. Finally, a noise term is added.

To find the peaks of this data, the mean amplitude of each waveform with an added small constant (the "noise gate") is taken. For each amplitude-time data point, if this amplitude is 1 order of magnitude greater than the noise gate and than those of the points before and after it, it is labelled as a peak. Once all data points have been inspected, the number of peaks found is output as a time-domain waveform graph.

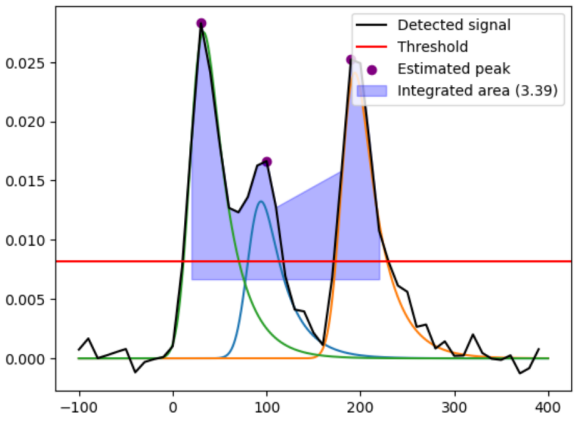
This algorithm combines the traditional approach of looking for differential maxima in the data set with a noise gate and an order of magnitude gate to filter out stochastic maxima in the noise region. During the development stages, the algorithm was heuristically evaluated via human eye, having it generate and find peaks for approximately a hundred graphs at a time. Two models were found to have the most success; the first one (the "sensitive approach") was to use the differential maximum and noise gate criteria only. Using the sensitive peak finding algorithm reliably marks every non-noise peak as a peak, however owing to the sometimes dramatic fluctuations in noise, falsely label some noise as a peak. The second one (the "double-gated approach") adds the order of magnitude gate and raises the noise gate, which reliably filters out all false positives from noise, but struggles to label low-amplitude peaks when they do not manage to exceed an order of



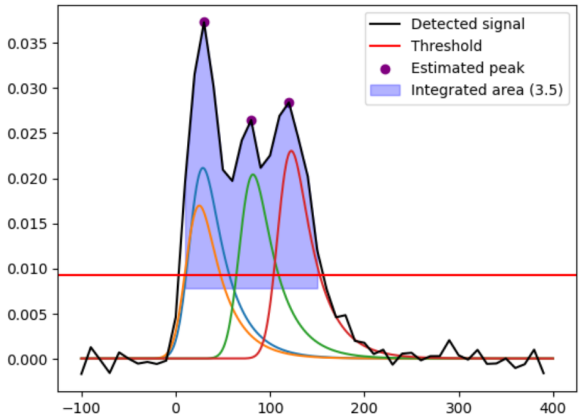
(a) A successful one photon trial.



(b) One of two photons was not found due to the order of magnitude cutoff.



(c) A successful three photon trial.



(d) Two of the photon waveforms interfere, making four photons appear to be three peaks.

Figure 4: An example trial of the double-gated peak finder for 1 through 4 photons.[Made in Python]

magnitude's difference to the noise. These design flaws are mutually exclusive, so one of the two models must be adopted.

Additionally, due to the primary driver of the peak finder being differential maxima, both models have the flaw of being unable to separate out PMTs which overlap too closely and appear to form only a single peak. Various modifications to the program were trialled, such as a context sensitive peak labeller that was able to label up to two adjacent points which had noticeably greater amplitude than all surrounding points. The reason none of these worked as intended is that the photon simulation is a stochastic process for which the algorithm has no context - for example, it does not know the actual amount of photons simulated like the programmers do, nor does it have an intrinsic idea of how many photons are expected for any given iterations of the simulator. Finally, neither the human eye nor any variant of the peak finder was able to truly tell how many photons comprised a

waveform with only one peak where the simulation data itself was obfuscated.

Thus, the peak finder had, in addition to the sensitive and double-gated variants, a fundamental flaw in being unable to accurately discern the nature of overlapping peaks. As LZ and other experiments approach the neutrino fog, their sensitivity to minute details must increase; as this difficulty is one of being able to determine how many of a certain label exists in a data set according to a set of criteria, machine learning, which specialises in such tasks, is thought to be applicable to this problem.

3 Machine Learning

3.1 Neural Networks

A neural network (NN) is a type of ML model designed to emulate a biological brain in an effort to make a self-improving machine capable of solving problems that have no conventional computational solution[19]. NNs are assembled in two steps, and must be calibrated in three; these are the training, evaluation, and testing steps[20].

The training step begins with forward propagation, where the NN accepts an input numerical data set and passes this data through its layers, performing transformations on the data in an attempt to extract patterns. The results of this procedure are expressed as an output such as predicting which one of a given set of classes a given data point belongs to. The data set contains the true classes for each data point, but this information is obfuscated during the forward step. Certain functions known as "activation functions"[21] are used in the forward step to calculate the output of a transformation in a brain-like way and add nonlinearity to the model, which is what enables NN learning. During the forward step, the NN passes data between "layers", which are sequences of operations that transform the input matrix and return an output matrix. The NN itself is a sequence of layers, which are executed one after the other, applying transformations the programmer believes are suited to extracting information from the given type of input data; the exact transformations depend on the task, but can include for example a simple matrix multiplication for binning data in a histogram, or a Fourier convolution for sound engineering. At each step in each layer, the NN passes the data between an array of "nodes", which defines the size of each "adjacent" dimension at each matrix multiplication step. Each node at each step is connected to every node in the next step in a web that resembles a brain. Furthermore, each node has a weight associated with it, which in this case determines how much importance to assign that node's overall impact on the problem at large. It is these weights that gives NNs the ability to self-improve, and finding the right value for each weight is a vital part of any ML model.

This is followed by backpropagation[22], where the NN goes finish to start through its programming and compares its output and decision making at each layer to the true analogous result in the data set. It does this by applying a loss function[23] which tracks the difference between its prediction and reality to its decisions, and an optimizer[24], which adjusts its performance according to this difference. During the backward step, the NN

goes backwards through its decision tree and evaluates the impact that each node's weight had for each transformation performed; at each step of the process, it compares its result to the no longer obfuscated true value and therefore all decisions that led to this value. The exact magnitude of the mistake made at each point is quantified through a "loss function", a mathematical function which computes the suboptimality of a given determined decision tree by computing the difference between the known best outcome and the outcome that was chosen. In a NN, this is implemented by finding the gradient between the true and predicted values with respect to the chosen loss function. Following this, the optimizer adjusts the weights at each node to get closer to minimizing the loss function at that node until the model eventually converges on a stable gradient.

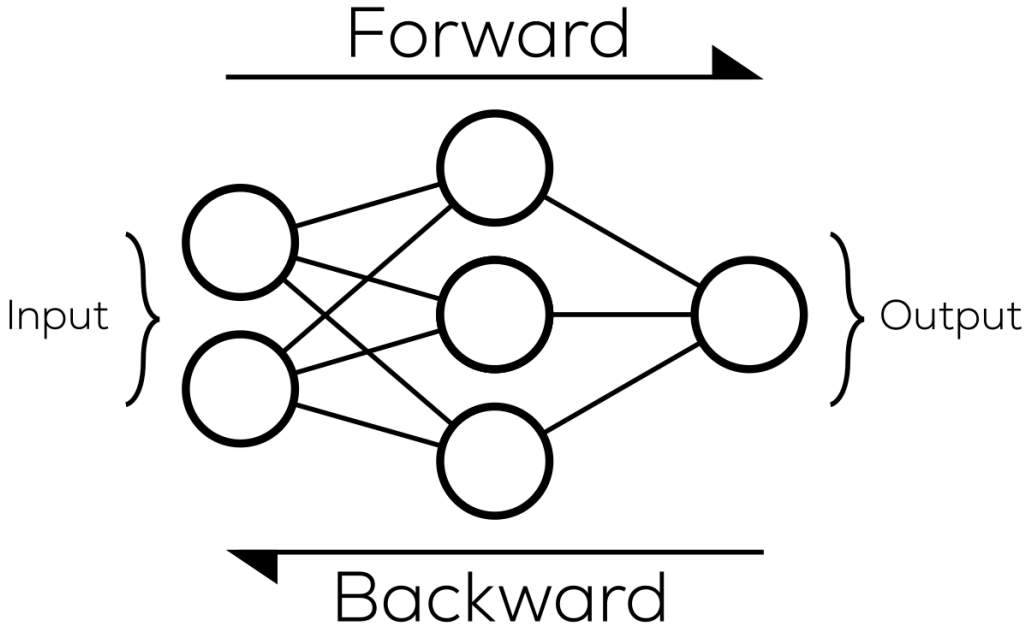


Figure 5: A visualisation of training a neural network with 2 input nodes, 3 hidden nodes, and 1 output node. One forward and one backwards pass are made per batch of data, and this repeats until all assigned batches and epochs have been iterated through.[Made in Inkscape.]

Once these steps have been carried out and a satisfactory weighting and minimization of bias is found, the NN can then be verified and used on data sets it has not been previously exposed to. In this case, only the forward step is used, and the end result of this step is the result of the overall application of the NN to the given data set.

3.2 Training And Evaluating Neural Networks

NNs are trained through exposure to mass amounts of data of a given type so that they can learn recurring patterns, or "features"[25], and look for those same combinations of features in future data sets. To do this, ML models are exposed to the data progressively

and iterate back over the data a significant number of times. This is done by separating this paradigm into two distinct steps. The data is separated into equally sized "batches" which the NN iterates through in sequence. The model repeats itself a number of times known as "epochs". Long-term memory units in the model preserve data between epochs, and at the end of each epoch, backpropagation occurs.[26]

The primary challenge to avoid when training a NN is "overtraining"[27], where the NN begins to learn features specific to the data set it is being trained on rather than features belonging to the domain of data it is analysing in general. Characteristic symptoms of an overtrained NN include almost perfect performance at the training data set with underwhelming performance when exposed to unseen data sets. The batch size can mitigate overtraining; each batch helps set the gradient computed at backpropagation. Too little of a batch size can prevent the gradient from converging, while a batch size too great can prevent a good estimate of the average gradient. Additionally, learning happens at the end of each epoch, and the more epochs a ML model is allowed to continuously operate on the same data set, the more it will acclimate to the features of that data set specifically. Therefore, it is important to find the right amount of epochs such that the model is allowed ample time to learn the data set's features, while not having too many epochs to where the model overtrains. Once trained, the NN must be evaluated to ensure proper performance

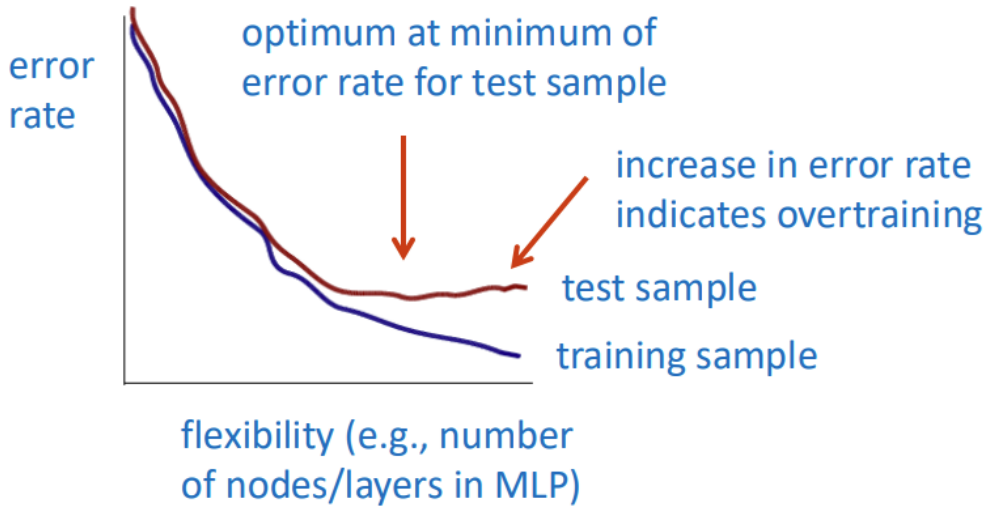


Figure 6: An example error-flexibility graph. Overtraining typically occurs following the inflection.[27]

on the full training data set without the help of backpropagation, then it must be tested on at least one unseen data set to ensure the model's applicability to previously unseen data. Traditionally, this is done by shuffling the full data set intended to be used on the model, then separating 20% of the full data set into a "test" data set, with the remaining 80% going towards a "training" data set[Acknowledgements]. This training data set is then further divided into five sub-sets. During training, the model is trained on the full set minus one sub-set at a time, and is then evaluated against the full training data set. Finally, it is tested against the unseen test data set. All of this is done to minimize the chances of the

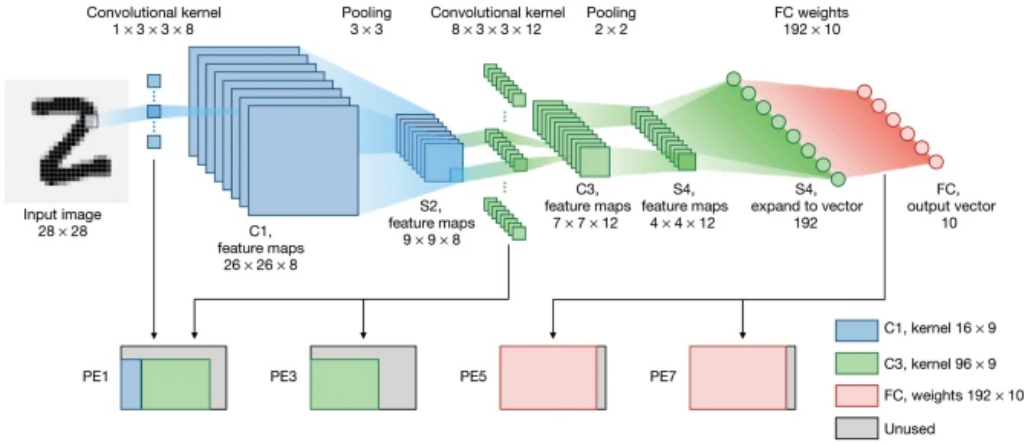


Figure 7: Architectural diagram of a convolutional neural network.[30]

model becoming too acclimated to any one pattern specific to the data set by introducing multiple sources of random variance.

The training of a NN is a stochastic process, hence best practice is to attempt to train and test the model repeatedly to find the average best epoch. However, the functions chosen for backpropagation matter greatly; a "learning rate" is associated with the optimizer, which determines the magnitude of weight adjustment during backpropagation steps, and several loss functions exist. Further methods to attempt to correct a poorly-trained ML model include de-noising the data set[28], and adding "dropout"[29] layers which randomly disable certain nodes in order to force the ML model to keep most if not all nodes active rather than relying on a select few, which can quickly lead to overspecialization. Finally, a large amount of data is required to train NNs, which can be inhibited by the availability of computing power and storage to the programmer.

Past a certain point, NNs have a tendency to descend into "black box"-style operation, meaning that any statements about how a certain NN reaches its goal can only ever approach definitiveness and are never properly definitive. Nevertheless, the project aims to look through the black box as much as possible.

3.3 ResNet and RawNet

RawNet[31], so named because it is a NN which analyses raw waveforms in the time-domain without converting them to Fourier space first, is a ML model which was designed for audio processing work. RawNet itself is an implementation of the residual neural network (ResNet) protocol[32], a machine learning protocol designed in 2015 to solve the "vanishing gradient problem"[33], which was a problem that arose in convolutional neural networks (CNNs) and similar models before then. A CNN is a type of ML model designed to imitate the human visual cortex for tasks such as image processing, which is achieved by including convolution operations as the foundation of the data processing algorithms[34]. This architecture, when extended to enough convolve layers, encounters the vanishing gradient problem, where due to repeated modification of the weights according to partial derivatives

of the loss function, these modifications would combine in a cascading way which eventually eliminated the gradient in the affected neurons entirely, impeding or outright preventing further learning and functioning of the NN. ResNet solves this problem by adding a "resid-

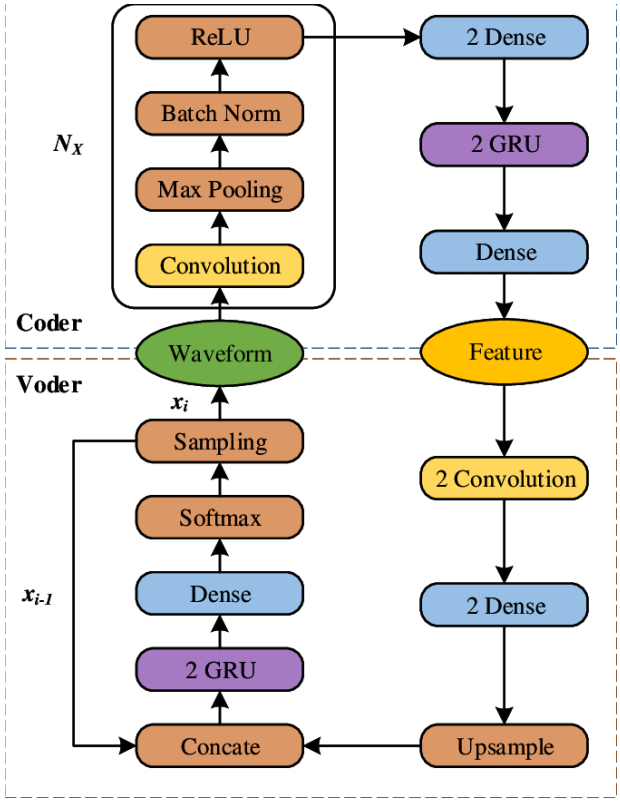


Figure 8: Architectural diagram of the RawNet vocoder.[35]

ual term" to the output of each convolve layer equal to its input, which adds a form of flat magnitude to each neuron, making them resilient to gradual degradation of their gradients due to large NN size. With the ResNet protocol in place, the basic idea behind a CNN is to convolve the input to extract features, then normalise and pool what remains to learn more and more abstract patterns as the model trains itself. RawNet implements this protocol with the goal of making a vocoder[36], which is a piece of software that deconstructs the sonic components of a given sound (the "coder") and uses them to modify the input sound or otherwise reconstruct other sounds (the "voder"). Though this is not in itself an image processing or computer vision task, the RawNet architecture finds its place in this role; analysis of an image converted to data form via convolution can be compared to the spectral deconstruction of a time-domain waveform via convolution. The RawNet architecture is as follows:

Layer	Functions
Strided	Conv1d(128,128,3) BatchNorm1d(50) LeakyReLU()
ResSmall x2	Conv1d(128,128,3) BatchNorm1d(50) LeakyReLU() MaxPool1D(3) Runs twice, has residual term
ResBig x2	Conv1d(256,256,3) BatchNorm1d(50) LeakyReLU() Conv1d(256,256,3) BatchNorm1d(50) LeakyReLU() MaxPool1D(3) Runs twice, has residual term
GRU	GRU(1024)
FC	Linear(1024,128) Linear(128,10)

Table 1: The programmatical structure of the modified RawNet for JUNO data processing. For convolutional layers, numbers inside parentheses refer to input size, output size and kernel size. For MaxPool layers, numbers inside parentheses refer to the dilution factor. For gated recurrent unit (GRU) and fully-connected layers, numbers inside the parentheses indicate the number of nodes.[37]

3.4 RawNet For Particle Physics

Owing to the success of using image recognition technology for sound engineering via the RawNet model, it stands to reason that RawNet may be usable for other purposes. Following this logic, it has been trialled for other purposes, including analysis of interactions in particle physics. A modified version of the voder half of the original RawNet model was trialled on simulated data recreating the inner workings of the JUNO experiment, with the goal of classifying how many of 0 through 9 photons comprise a given waveform. The forward architecture of the modified RawNet is as follows[37]:

There are multiple reasons for such dramatic modifications. Firstly, since this is only a classification task, only data extraction needs to occur, meaning only the coder half of RawNet is useful to this problem. Secondly, the convolve layers were changed to involve a second convolution operation and one less activation function in order to further separate out the involved ten generations of particles. Finally, more maximum pooling layers were introduced, however owing to the use of only four residual layers which minimises the impact of the vanishing gradient problem, only one GRU was used. The fully connected layer

was additionally modified to output 10 neurons, which each correspond to the probabilities of the analysed events being one of the ten given classes.

This modified RawNet performed very well for photon-waveform discrimination for neutrino flavour oscillation experiments, so it would make sense to pursue a modification of the architecture for the LZ problem.

3.5 Adapting RawNet to This Problem

It was noticed that the LZ simulator outputs data in a raw time-domain format similar to the time-domain waveforms the RawNet models operate on. In theory, it would be possible to train a RawNet offshoot on the LZ data with modifications made to improve it for the task of analysing a single low-complexity particle track with the express intent of classifying how many photons the event released.

This was attempted, and is the main focus of this report. The architecture of the modified model, named ExactDarkEvaluationNet (EDEN) due to its operating on unmodified time-domains and focus on smaller scale problems in dark matter, is as follows, beginning with the forward step:

Layer	Functions
Strided	Conv1d(50,50,3) BatchNorm1d(50) LeakyReLU()
ResSmall x2	Conv1d(50,50,3) LeakyReLU() Conv1d(50,50,3) LeakyReLU() Dropout(0.02) Runs twice, has residual term
ResBig x2	Conv1d(50,100,3) LeakyReLU() Conv1d(100,50,3) LeakyReLU() Conv1d(100,50,3) LeakyReLU() Conv1d(50,100,3) LeakyReLU() Dropout(0.04) has residual term
GRU	GRU(50,200)
FC	Linear(200,800) ReLU() Linear(800,8) Sigmoid()

Table 2: The programmatical structure of EDEN. For convolutional layers, numbers inside parentheses refer to input size, output size and kernel size. For gated recurrent unit (GRU) and fully-connected layers, numbers inside the parentheses indicate the number of nodes. For dropout layers, numbers inside the parentheses indicate the percentage of the NN's nodes to randomly disable, expressed as a fraction.

Conv1d performs a convolution of the y-axis values along the x-axis, allowing the breakdown of a waveform to determine its constituent features.
BatchNorm1d layer normalizes all input data to a value between 0 and 1 to prevent dramatic gaps between the values of adjacent data points from too heavily distorting the NN's weighting.
LeakyReLU is activation function. Positive inputs are linearised, while negative inputs have a very small but non-zero gradient to prevent vanishing at high scales.
Dropout randomly disables the given percentage of the neural network's nodes to mitigate overtraining.
The Gated Recurrent Unit is a memory retention protocol which maps incoming information to either be retained for the next epoch or discarded by then, based on its perceived value.
Linear performs a matrix multiplication to determine the relationship between all incoming nodes as an output scalar $y = ax + b + cx + d + ...$, factoring in biases.
ReLU is an activation function. Positive inputs are linearised, while negative inputs are annulled.
Sigmoid is an end step function which decomposes the model's result into a list of confidences that the model has in the given data being of each given class; these confidences must sum to 100%.

For backpropagation, EDEN uses the cross-entropy loss function, which expresses the difference between the predicted and true outcomes in the form of a logarithmic relationship, and the Adam optimiser with a starting learning rate of 0.0001, which tracks the average exponential decay of past gradient updates. This is used to determine the impact that adjusting any given weight has, as well as dynamically modifying the learning rate according to how noisy the data is becoming.

Like the JUNO-simulation RawNet derivative from which it derives, EDEN is a classifier, which analyses an incoming time-domain waveform and attempts to classify which class of "number of contributing PMTs" the waveform belongs to. Compared to the JUNO-simulation RawNet derivative, EDEN further leans into the simplicity of the model to account for the simplicity of the task at hand. The convolve layers remain, however all examples of batch normalization save for the very first one and maximum pooling have been removed, as the data used to train the model already contains very few points, and repeated normalization quickly degreades the data. The GRU layer remains, but has been reduced in scope owing to the lesser amount of memory needed to be retained. As this is a small scale problem with a small data set, LeakyReLU is used to prevent early loss of mass amounts of nodes, and Sigmoid is used to return classification probabilities for the final evaluation. Dropout layers were added when it was found that the initial version of EDEN was prone to overtraining at any step of learning.

Overall, EDEN has trouble learning the patterns required to classify time-domain waveforms sourced from LZ data. The results are discussed in further detail below. One notable

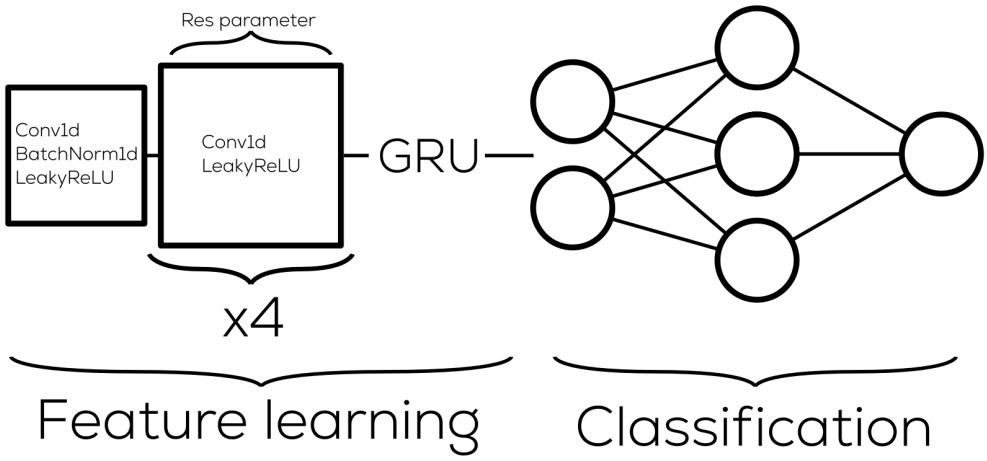


Figure 9: An architectural diagram of EDEN.[Created in Inkscape]

weakness of the model unrelated to its performance is that it has to be "prepared" beforehand; the convolutional layers can only accept one size of batched data input, so all input data must be run through a batching program that separates it into batches of the given size. Additionally, the number of output nodes of the fully connected layer and target classes assigned to the CrossEntropy layer must match, so the theoretical minimum and maximum possible classes the model will distinguish between must be calculated and hard-coded into the model for each use. EDEN was implemented through a relatively simple algorithm using PyTorch.

4 Performance

4.1 Conventional

Both the sensitive and double-gated conventional peak finding algorithms were trialled against a hundred simulated waveforms with anywhere between 1 to 4 photons each. The algorithms were used to label the amount of predicted peaks; as a sanity check, the area under the curve of each identified peak was automatically calculated by using NumPy's trapezoid method function, with the time-axis points where the amplitude becomes equal to the noise gate used as the horizontal limits. Each photon should roughly contribute an area of 1 to this calculation. The area under the curve calculation was roughly found to line up with the found peaks in the peak finder, so work could proceed. As a stress test, the algorithms were also trialled against simulated waveforms with anywhere between 0 to 7 photons each, but they never correctly identified more than 8% of the data populations above 5 photons. It should be noted that seeing more than 3 photons in one PMT is highly improbable. To quantify the accuracy of the peak finder, the true number of contributing photons was saved alongside each waveform but obfuscated from the peak finder. For

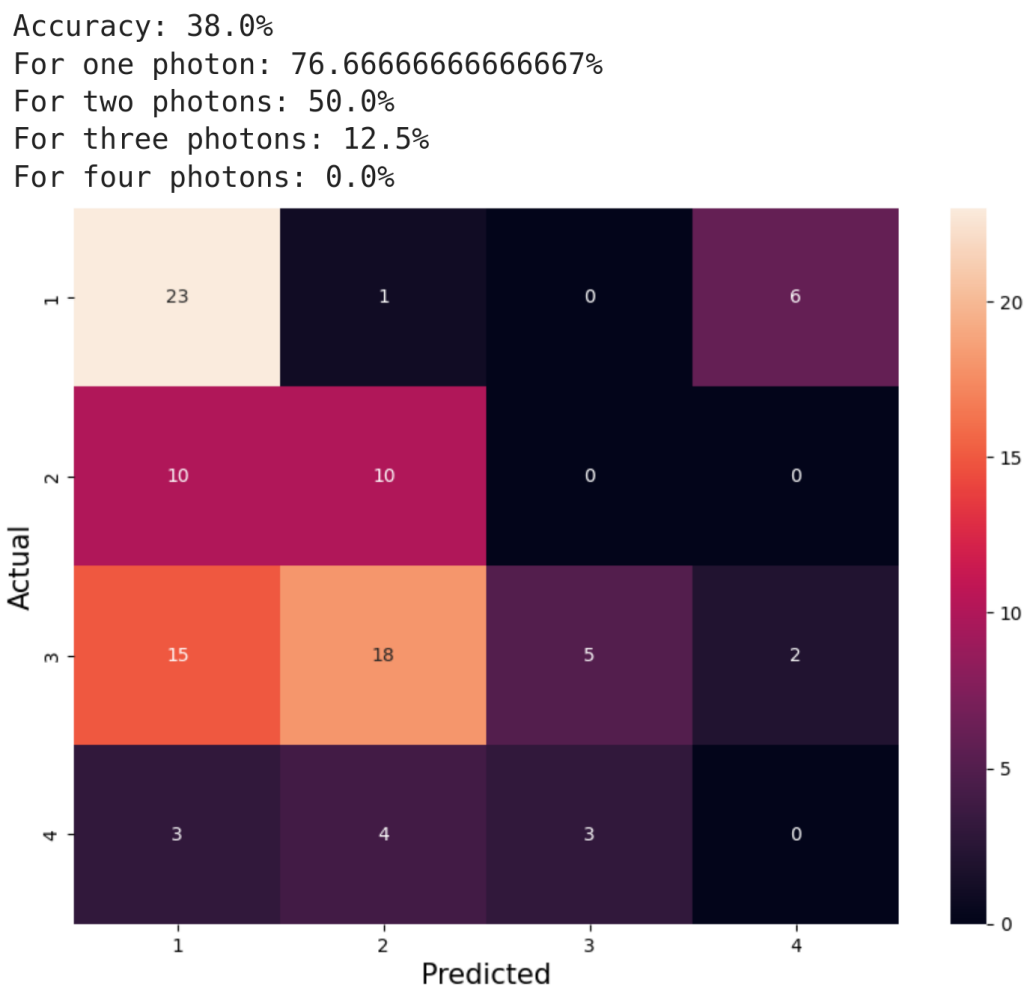
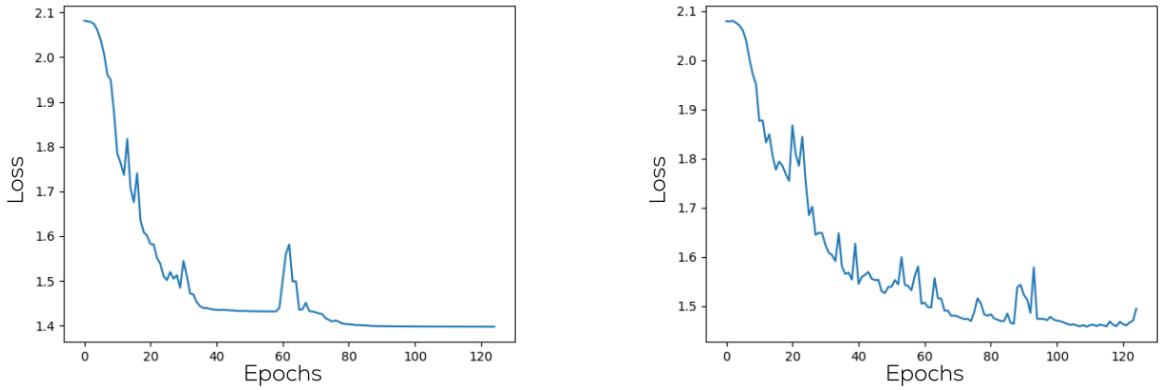


Figure 10: A confusion matrix measuring the conventional peak finder's performance for 1 through 4 photons.[Made in Python]

each waveform analysed, the peak finder's guess was tabulated alongside its true value in a "confusion matrix", which is an array normally used to evaluate the performance of a classification ML model by comparing its prediction of a given data point's class to its actual class. It was observed that the confusion matrix structure was appropriate to this task. By tabulating the confusion matrix, it was shown that the peak finder correctly identified on average 40% of the photons, skewed heavily towards lower photon counts with it almost never correctly identifying four photons. This was used as the baseline to compare EDEN to. An alternate peak finding method is the existing technology used in the LZ, where the integrated area under the curve is directly used to classify the amount of peaks. For this method, the area is plotted as a Gaussian with each separate photon contributing a peak and each overlapping photon contributing about 1σ to the Gaussian's width. This method is better at classifying high photon counts, but can encounter problems when classifying neighbouring photon values, as showcased in Figures 6a and 6b where the integrated areas are well within 1σ of each other.

4.2 EDEN

To determine the ideal amount of epochs for which to train EDEN, the training and validation steps were run repeatedly, occasionally changing the epoch number, and plotted as a function of loss against epochs to determine how EDEN learns with time. The results found by the model during its evaluation step were plotted as a confusion matrix; given the nature of the problem as a classification problem, this provides a quick way to visually evaluate the model's performance by looking at how diagonal the heat map is. Using



(a) (b)

Figure 11: EDEN's loss plotted as a function of the epochs passed. This curve shows erratic behaviour in the model, where it continuously converges but never to a degree where an obvious point of convergence exists. Additionally, spikes exist in its loss, which may indicate points of overtraining. Note the difference in the two curves despite being trained on the same data set, showing the stochastic nature of ML. Made in PyTorch and Inkscape

125 epochs and a batch size of 200, when applied to previously unseen test data, EDEN's performance was as follows:

To test the model's limits, EDEN was tested up to unrealistic contributing PMT counts of up to 7, and with some data points having zero simulated PMTs to test its ability to correctly identify pure noise.

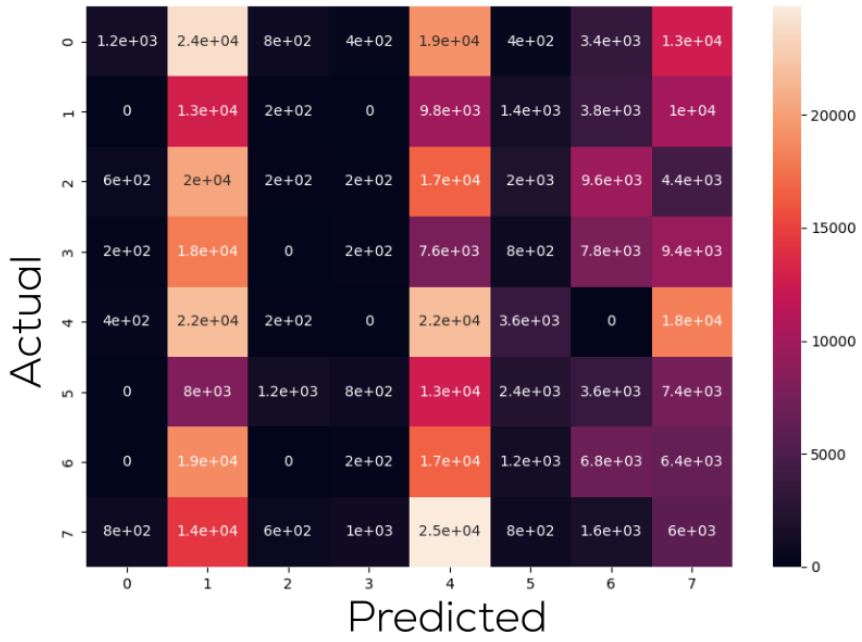


Figure 12: A confusion matrix of EDEN's performance when tasked to classify waveforms between 0 through 7 contributor photons for data it has not previously seen. Note its tendency to favour approximating incoming data towards two or three "favoured" classes.[Made in PyTorch and Inkscape]

4.3 Comparisons and Progression

Compared to the peak finding baseline, EDEN seemed to encounter similar problems, only displaced by class. Whereas the conventional peak finder leaned towards classifying all inputs as low photon counts, EDEN appeared to learn the patterns for two or three photon counts and then classify every incoming waveform according to which of these photon counts they most closely resembled; the peak finder could be interpreted as a specialised case of this phenomenon, with the favoured classes being 0, 1, and 2 photons. This would seem logical, as CNNs were first implemented for the purposes of computer image recognition, and it is difficut to decipher the amount of photons in a given waveform in LZ data by eye; EDEN may be encountering the same problem. This may also be a result of having to strip out the BatchNorm1d() and MaxPool1d() functions to allow the model to learn without diluting the data to zero, as batch normalization and max-pooling are important parts of CNNs.

EDEN is also cumbersome to wield in ways that limit the potential of this exact implementation. The biggest challenge to EDEN's performance is its volatility; the model very readily overtrains, and its loss-epoch graph is far more stochastic than the average for neural networks. The normal strategy for determining the optimal epochs of a NN is to plot a loss-epoch graph and identify the inflection point, however EDEN's loss-epoch

graphs fluctuate wildly with an overall downward trend that can last for hundreds of epochs. However, heuristic observations during testing identified that the model reaches overtraining well before the final flattening of the training curves; owing to the relative lack of information conveyed by the training curves, the ideal epochs to train the model had to be found heuristically via trial and error. Owing to these challenges and the fact that EDEN is a simple refitting of the JUNO-derivative RawNet model with little divergence from its template, it is especially possible that the hyperparameters used in this project are nowhere near optimal for EDEN's performance. This version of EDEN may well remain suited only for photon counting on JUNO data, and further analysis in a different project may be able to find a better performing version of the technology.

Another challenge to the model's performance is its rigidity. As a CNN, EDEN relies on the Conv1d function to do the bulk of the feature-identifying work, and the Conv1d function demands a specific input and output shape for its matrices, as well as the FC layer. This means that the input data is essentially restricted to only having a certain amount of data points, otherwise the model does not function. Furthermore, as a classifier, EDEN can only fit unseen data to one of an upper limit of classes without program-threatening software errors, which limits the scope of problems that it can be applied to, as application of the model to fit a potentially unknown maximum number of classes may prove impossible. Despite these limitations, the model is certainly usable, but this requires extra hardcoding to ensure all incoming data is batched specifically into the accepted batch size, and extra preliminary research to discover the practical upper limit to the amount of classes it will be asked to discriminate between. Furthermore, because the batch size can be dependent on the type of data being gathered, needing to modify the batch size may hinder the model's performance. Finally, EDEN is not suited to complicated tasks or data already existing in Fourier space, owing to its overall lack of normalization functions rendering it less effective against multi-track problems and the strided layer being the starting convolution layer that primes the input for further analysis.

During testing, various iterations of the model were tried. Owing to the black box nature of NNs, the following is a presentation of quirks noticed in the model's operation deemed below standard and later ironed out, and theories deemed probable to explain them. Removing the GRU had a catastrophic effect on the model's performance in every regard, likely due to losing the long-term memory component. The BatchNorm1d and MaxPool1d layers had to be reduced in scope die to data dilution. Adding more Conv1d layers appeared to make the model worse at identifying features from the data it is given, but also caused it to treat test data a lot closer to training data than the focused final version that was tested. Switching from ReLU() to LeakyReLU() as the activation function had an overall beneficial effect, likely due to having more nodes intact and non-zero by the fully connected layer. Changing the amount of hidden nodes in the fully connected and GRU layers seemed to have little impact, as did changing the kernel size of the Conv1d layers. Both res layers must be present, as the small one performs baseline feature extraction while the big one with more nodes extends this to more complicated patterns. Finally, removing the residual term from the forward step made the model more vulnerable to vanishing gradient and decreased its learning rate and quality.

When improperly training the model by failing to shuffle the data sets or separate them

properly for training through testing, it was observed that the model performed excellently on both training and testing due to overtraining. Here, the volatility of its overtraining threshold was even more pronounced than usual, where it appeared to learn little until a range of two or three epochs where it suddenly became able to process the data set, after which it would rapidly overtrain.

Overall, EDEN in its current implementation is unable to learn the patterns needed to correctly discriminate between classes of photons, however it is shown to still be learning patterns in the data instead of making random guesses; what remains curious is how these patterns appear to correspond to the same difficulties the human eye and the existing peak finding algorithms face in this task. The volatility with which the model overtrains and the wild fluctuation of its loss-epoch graphs make it difficult to identify the exact point where it "understands" the exact nature of this relationship. Finally, during testing, adding more convolutions revealed an inverse correlation between success at the evaluation step and success at the testing step. The reason for this remains unknown, and provides an avenue for future research.

5 Conclusions

As DM searches approach the neutrino fog, the ability for experiments to discriminate between even slight differences in events becomes more important. Though conventional peak finders have thus far performed well in simulation trials, they begin to lose performance when tried with WIMP cross sections this close to the neutrino floor in either distinguishing actual peaks from noise or labelling low-intensity peaks that lie close to the noise based on the peak finding algorithm used. Furthermore, peak finders prove increasingly unable to account for overlapping peaks which superpose and appear to form one single peak, due to various factors such as the lack of information these peak finders have compared to the humans building them. To circumvent this, a machine learning model based off of the RawNet vocoder was trialled.

As RawNet is a system that breaks down an incoming waveform in the time-domain and analyses its features to later reconstruct them, and it was observed that most particle physics detection problems can be formulated as "time-domain waveform output to be deconstructed for identifying features", it would make sense that RawNet could be modified for use in such problems. A modified RawNet architecture was trialled against simulated JUNO data, where it was found to be excellent at classifying which of 0 through 9 photons a given waveform was comprised of. This modification was further modified into the ExactDarkEvaluationNet (EDEN) model analysed in the project, which failed to improve on the existing peak finders, but also showed similar problems to them despite the radically different approach to solving the problem. EDEN is simple and remains cumbersome, requiring hardcoding of the incoming batch size for any data to be applied to it and of the maximum possible amount of classes which the model is expected to discriminate between; this means that applying EDEN to a given problem requires thorough pre-existing knowledge of the problem's dynamics. However, it is still apparent that EDEN has the potential to learn patterns in raw time-domain data at low event complexities; given future

research and an opportunity to further diverge from the architecture it was built from, EDEN and related technologies may yet form the backbone of the machine learning-assisted breakthrough that LZ and similar experiments need to achieve the next step in detection sensitivity.

References

[1] F. Zwicky. "Die Rotverschiebung von extragalaktischen Nebeln". In: *Helvetica Physica Acta* 6 (Jan. 1933), pp. 110–127.

[2] James Binney and Scott Tremaine. *Galactic dynamics*. Princeton university press, 2011. Chap. 10.1.

[3] Wayne Hu and Scott Dodelson. "Cosmic Microwave Background Anisotropies". In: *Annual Review of Astronomy and Astrophysics* 40.1 (Sept. 2002), pp. 171–216. ISSN: 1545-4282. DOI: 10.1146/annurev.astro.40.060401.093926. URL: http://dx.doi.org/10.1146/annurev.astro.40.060401.093926.

[4] J. R. Brownstein and J. W. Moffat. "The Bullet Cluster 1E0657-558 evidence shows modified gravity in the absence of dark matter". In: *Monthly Notices of the Royal Astronomical Society* 382.1 (Oct. 2007), pp. 29–47. ISSN: 0035-8711. DOI: 10.1111/j.1365-2966.2007.12275.x. eprint: https://academic.oup.com/mnras/article-pdf/382/1/29/3057374/mnras0382-0029.pdf. URL: https://doi.org/10.1111/j.1365-2966.2007.12275.x.

[5] Katherine Garrett and Gintaras Duda. "Dark Matter: A Primer". In: *Advances in Astronomy* 2011 (2011), pp. 1–22. ISSN: 1687-7977. DOI: 10.1155/2011/968283. URL: http://dx.doi.org/10.1155/2011/968283.

[6] J I Read. "The local dark matter density". In: *Journal of Physics G: Nuclear and Particle Physics* 41.6 (May 2014), p. 063101. ISSN: 1361-6471. DOI: 10.1088/0954-3899/41/6/063101. URL: http://dx.doi.org/10.1088/0954-3899/41/6/063101.

[7] David JE Marsh, Viraf M Mehta, and David Ellis. "Dark matter: evidence, theory, and constraints". In: *Annual Review of Astronomy and Astrophysics* (2024).

[8] D.S. Akerib et al. "The LUX-ZEPLIN (LZ) experiment". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 953 (Feb. 2020), p. 163047. ISSN: 0168-9002. DOI: 10.1016/j.nima.2019.163047. URL: http://dx.doi.org/10.1016/j.nima.2019.163047.

[9] E. Aprile et al. "First Dark Matter Search with Nuclear Recoils from the XENONnT Experiment". In: *Phys. Rev. Lett.* 131 (4 July 2023), p. 041003. DOI: 10.1103/PhysRevLett.131.041003. URL: https://link.aps.org/doi/10.1103/PhysRevLett.131.041003.

[10] B. J. Mount et al. *LUX-ZEPLIN (LZ) Technical Design Report*. 2017. arXiv: 1703.09144 [physics.ins-det]. URL: https://arxiv.org/abs/1703.09144.

[11] Ben Carew et al. *The neutrino fog for dark matter-electron scattering experiments*. 2024. arXiv: 2312.04303 [hep-ph]. URL: https://arxiv.org/abs/2312.04303.

[12] J. Aalbers et al. “Dark Matter Search Results from ¡mml:math xmlns:mml=”http://www.w3.org/199 display=”inline”¿¡mml:mrow¿¡mml:mn¿4.2¡/mml:mn¿¡mml:mtext¿¡/mml:mtext¿¡mml:mtext¿¡/mml of Exposure of the LUX-ZEPLIN (LZ) Experiment”. In: *Physical Review Letters* 135.1 (2025). ISSN: 1079-7114. DOI: 10.1103/4dyc-z8zf. URL: http://dx.doi.org/10.1103/4dyc-z8zf.

[13] *Photomultiplier Tubes, Basics and Operation (Fourth Edition)*. Hamamatsu. 2016. URL: https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2ahUKEwiLnLjx8_iRAxURT0EAHf33FXcQFnoECAwQAQ&url=https%3A%2F%2Fwww.hamamatsu.com%2Fcontent%2Fdam%2Fhamamatsu-photonics%2Fsites%2Fdocuments%2F99_SALES_LIBRARY%2Fetd%2FPMT_handbook_v4E.pdf&usg=AOvVaw3hRIXFoe8lkNO6pn55vSFO&opi=89978449.

[14] Tom M. Mitchell. *Machine Learning*. 1997. URL: https://openlibrary.org/books/OL39827446M/Machine_Learning.

[15] Travis Oliphant. *NumPy documentation*. 2005. URL: https://numpy.org.

[16] Unknown. *Awkward documentation*. Unknown. URL: https://pypi.org/project/awkward/.

[17] Unknown. *hist documentation*. Unknown. URL: https://pypi.org/project/hist/.

[18] C. J. CLOPPER and E. S. PEARSON. “THE USE OF CONFIDENCE OR FIDUCIAL LIMITS ILLUSTRATED IN THE CASE OF THE BINOMIAL”. In: *Biometrika* 26.4 (Dec. 1934), pp. 404–413. ISSN: 0006-3444. DOI: 10.1093/biomet/26.4.404. eprint: https://academic.oup.com/biomet/article-pdf/26/4/404/823407/26-4-404.pdf. URL: https://doi.org/10.1093/biomet/26.4.404.

[19] Larry Hardesty. *Explained: Neural networks*. 2017. URL: https://news.mit.edu/2017/explained-neural-networks-deep-learning-0414.

[20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[21] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. *Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark*. 2022. arXiv: 2109.14545 [cs.LG]. URL: https://arxiv.org/abs/2109.14545.

[22] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: 323.6088 (Oct. 1986), pp. 533–536. DOI: 10.1038/323533a0.

[23] Y. Zhong A. Mao M. Mohri. *Cross-Entropy Loss Functions: Theoretical Analysis and Applications*. 2023. URL: https://arxiv.org/pdf/2304.07288.

[24] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[25] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. 2006. URL: https://openlibrary.org/books/OL7445337M/Pattern_Recognition_and_Machine_Learning_%28Information_Science_and_Statistics%29.

[26] Li Yang and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice". In: *Neurocomputing* 415 (Nov. 2020), pp. 295–316. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2020.07.061. URL: http://dx.doi.org/10.1016/j.neucom.2020.07.061.

[27] Glen Cowan. *Statistical Data Analysis 2025/26 Lecture Week 5*. Royal Holloway University of London, 2025. URL: https://www.pp.rhul.ac.uk/~cowan/stat/lectures/stat_week_5.pdf.

[28] Shivani Gupta and Atul Gupta. "Dealing with Noise Problem in Machine Learning Data-sets: A Systematic Review". In: *Procedia Computer Science* 161 (2019). The Fifth Information Systems International Conference, 23-24 July 2019, Surabaya, Indonesia, pp. 466–474. ISSN: 1877-0509. DOI: https://doi.org/10.1016/j.procs.2019.11.146. URL: https://www.sciencedirect.com/science/article/pii/S1877050919318575.

[29] David Warde-Farley et al. *An empirical analysis of dropout in piecewise linear networks*. 2014. arXiv: 1312.6197 [stat.ML]. URL: https://arxiv.org/abs/1312.6197.

[30] P. et al. Yao. "Fully hardware-implemented memristor convolutional neural network". In: *Nature* (Jan. 2020). DOI: https://doi.org/10.1038/s41586-020-1942-4. URL: https://www.nature.com/articles/s41586-020-1942-4.

[31] Jee-weon Jung et al. *RawNet: Advanced end-to-end deep neural network using raw waveforms for text-independent speaker verification*. 2019. arXiv: 1904.08104 [eess.AS]. URL: https://arxiv.org/abs/1904.08104.

[32] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

[33] Sunitha Basodi et al. *Gradient Amplification: An efficient way to train deep neural networks*. 2020. arXiv: 2006.10560 [cs.LG]. URL: https://arxiv.org/abs/2006.10560.

[34] Yann Lecun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *Nature* 521.7553 (May 2015), pp. 436–444. DOI: 10.1038/nature14539. URL: https://hal.science/hal-04206682.

[35] Yunchao He and Yujun Wang. *RawNet: Fast End-to-End Neural Vocoder*. 2023. arXiv: 1904.05351 [eess.AS]. URL: https://arxiv.org/abs/1904.05351.

[36] Homer W Dudley. *System for the artificial production of vocal or other sounds*. 1937. URL: https://patents.google.com/patent/US2121142A/en.

[37] Wei Jiang et al. *Machine-Learning based photon counting for PMT waveforms and its application to the improvement of the energy resolution in large liquid scintillator detectors*. 2024. arXiv: 2405.18720 [physics.ins-det]. URL: https://arxiv.org/abs/2405.18720.

A Acknowledgements

Amy McCullough - a quick lesson in machine learning that greatly improved the fidelity of the tests run on EDEN
Owen Cottle - consultation and theoretical help for the duration of the project